

# Auto-CRUD Generator (Option B) — .NET Framework 4.7.3

This document contains a complete **Auto-CRUD Generator** that *writes files into your MVC project* following your Religion sample pattern. Drop the generator into your solution, call the Generate endpoint (admin-only), and it will create the folder structure and files for any entity you specify.

**Important:** This generator executes file I/O and can execute DDL if you enable it. Only admins should be allowed to use it. Always test on a dev DB first.

---

## Folder structure produced (example)

```
/AutoCrudGenerator
  AutoCrudController.cs
  AutoCrudGenerator.cs
  Models
    AutoCrudInput.cs
    AutoCrudField.cs
    AutoCrudResult.cs
  Templates
    ModelTemplate.txt
    BLTemplate.txt
    DATemplate.txt
    ControllerTemplate.txt
    ViewTemplate.txt
    SqlTemplate.txt
/Generated <-- runtime output (one folder per entity)
/Religion
  Religion.BE.cs
  ReligionDB.DA.cs
  Religion.BL.cs
  ReligionController.cs
  tbl_Religion_Create.sql
  tbl_Religion_Procedures.sql
  Religion.Index.cshtml
```

---

## How it works (high level)

1. Admin opens the generator UI (or calls the controller endpoint) and posts JSON metadata (TableName, Fields).

2. `AutoCrudGenerator` builds SQL and C# source strings using templates.
3. Files are written to `~/Generated/{TableName}/` (you can change the output path).
4. Optionally, the controller can execute the SQL script against the configured DB — disabled by default for safety.

---

## 1) Models (`AutoCrudInput`, `AutoCrudField`, `AutoCrudResult`)

File: `AutoCrudGenerator/Models/AutoCrudInput.cs`

```
using System.Collections.Generic;

namespace AutoCrudGenerator.Models
{
    public class AutoCrudField
    {
        public string FieldName { get; set; }
        public string SqlType { get; set; } // e.g. nvarchar(200), int,
datetime, bit, decimal(18,2)
        public bool IsPrimaryKey { get; set; }
        public bool IsNullable { get; set; }
        public bool IsUnique { get; set; }
    }

    public class AutoCrudInput
    {
        public string Module { get; set; } // e.g. Master
        public string TableName { get; set; } // logical name, e.g. Religion
        public bool UseTablePrefix { get; set; } // default true ->
tbl_TableName
        public List<AutoCrudField> Fields { get; set; }
        public bool ExecuteSql { get; set; } // default false - requires
confirmation to run
        public string OutputFolderRelative { get; set; } // e.g. "~/Generated/"
    }

    public class AutoCrudResult
    {
        public string CreateTableSql { get; set; }
        public string ProceduresSql { get; set; }
        public string ModelCode { get; set; }
        public string DAcode { get; set; }
        public string BLcode { get; set; }
        public string ControllerCode { get; set; }
        public string ViewCode { get; set; }
        public bool IsSuccess { get; set; }
    }
}
```

```

        public string Message { get; set; }
    }
}

```

## 2) AutoCrudGenerator (core generator)

File: AutoCrudGenerator/AutoCrudGenerator.cs

```

using System;
using System.IO;
using System.Linq;
using System.Text;
using AutoCrudGenerator.Models;
using System.Collections.Generic;

namespace AutoCrudGenerator
{
    public class AutoCrudGenerator
    {
        private static string SafeName(string s)
        {
            if (string.IsNullOrEmpty(s)) return s;
            var clean = new string(s.Where(c => char.IsLetterOrDigit(c) || c ==
'_'').ToArray());
            return clean;
        }

        private static string TableNameWithPrefix(AutoCrudInput input)
        {
            var name = SafeName(input.TableName);
            return input.UseTablePrefix ? ("tbl_" + name) : name;
        }

        public AutoCrudResult GenerateAll(AutoCrudInput input)
        {
            var r = new AutoCrudResult();
            try
            {
                // Basic validation
                if (input == null) throw new ArgumentNullException("input");
                if (string.IsNullOrEmpty(input.TableName)) throw new
ArgumentException("TableName required");
                if (input.Fields == null || !input.Fields.Any()) throw new
ArgumentException("At least one field required");
            }
            catch { }
        }
    }
}

```

```

        // Build SQL
        r.CreateTableSql = BuildCreateTable(input);
        r.ProceduresSql = BuildProcedures(input);

        // Build C# code
        r.ModelCode = BuildModel(input);
        r.DAcode = BuildDA(input);
        r.BLcode = BuildBL(input);
        r.ControllerCode = BuildController(input);
        r.ViewCode = BuildIndexView(input);

        r.IsSuccess = true;
        r.Message = "Generated in-memory successfully";
    }
    catch (Exception ex)
    {
        r.IsSuccess = false;
        r.Message = ex.Message;
    }
    return r;
}

#region SQL Builders
private string BuildCreateTable(AutoCrudInput input)
{
    var tbl = TableNameWithPrefix(input);
    var sb = new StringBuilder();
    sb.AppendLine("IF OBJECT_ID(N'" + tbl + "', 'U') IS NOT NULL");
    sb.AppendLine("    DROP TABLE [" + tbl + "];\nGO\n");

    sb.AppendLine("CREATE TABLE [" + tbl + "] (");

    // PK
    var pk = input.Fields.FirstOrDefault(f => f.IsPrimaryKey);
    var pkName = pk != null ? pk.FieldName : input.TableName + "Id";
    sb.AppendLine($"    [{pkName}] INT IDENTITY(1,1) NOT NULL PRIMARY
KEY,");

    // columns
    foreach (var f in input.Fields.Where(x => !x.IsPrimaryKey))
    {
        var nullTxt = f.IsNullable ? "NULL" : "NOT NULL";
        var uniqueTxt = f.IsUnique ? " UNIQUE" : string.Empty;
        sb.AppendLine($"    [{f.FieldName}] {f.SqlType} {nullTxt}
{uniqueTxt},");
    }
}

```

```

        // standard metadata
        sb.AppendLine("    CreateBy INT NOT NULL CONSTRAINT fk_" +
input.TableName + "_CreateBy FOREIGN KEY REFERENCES TBL_USER(UserId),");
        sb.AppendLine("    ModifyBy INT NULL CONSTRAINT fk_" +
input.TableName + "_ModifyBy FOREIGN KEY REFERENCES TBL_USER(UserId),");
        sb.AppendLine("    UpdateLogDateTime DATETIME NULL,");
        sb.AppendLine("    LogDateTime DATETIME NOT NULL CONSTRAINT
DF_tbl_" + input.TableName + "_LogDateTime DEFAULT (GETDATE())");
        sb.AppendLine(");\nGO\n");

        return sb.ToString();
    }

    private string BuildProcedures(AutoCrudInput input)
    {
        var tbl = TableNameWithPrefix(input);
        var pk = input.Fields.FirstOrDefault(f => f.IsPrimaryKey);
        var pkName = pk != null ? pk.FieldName : input.TableName + "Id";
        var nonPk = input.Fields.Where(x => !x.IsPrimaryKey).ToList();

        var sb = new StringBuilder();

        // Add
        sb.AppendLine("CREATE PROCEDURE usp_Add" + input.TableName);
        sb.AppendLine("(");
        foreach (var c in nonPk)
            sb.AppendLine($"@{c.FieldName} {c.SqlType}{(c.IsNullable ? " =
NULL" : string.Empty)},");
        sb.AppendLine("@UserId INT ,");
        sb.AppendLine("@EntityId INT = NULL ,");
        sb.AppendLine($"@{pkName} INT = NULL OUTPUT ,");
        sb.AppendLine("@ResponseMSG NVARCHAR(254) = 'Invalid' OUTPUT ,");
        sb.AppendLine("@IsSuccess BIT = 0 OUTPUT, ");
        sb.AppendLine("@ErrorNumber INT = 0 OUTPUT");
        sb.AppendLine(")");
        sb.AppendLine("AS");
        sb.AppendLine("BEGIN");
        sb.AppendLine("    SET NOCOUNT ON;\n    EXEC sp_set_session_context
@key=N'UserId', @value=@UserId;\n");

        // optional duplicate check on Name
        if (nonPk.Any(x => x.FieldName.Equals("Name",
StringComparison.OrdinalIgnoreCase)))
        {
            sb.AppendLine("    IF EXISTS(SELECT 1 FROM dbo." + tbl +
" (NOLOCK) T WHERE T.Name = @Name)");
            sb.AppendLine("    BEGIN");
            sb.AppendLine("        SET @IsSuccess = 0;");
        }
    }

```

```

        sb.AppendLine("        SET @ResponseMSG = '' + input.TableName
+ " already exists.';");
        sb.AppendLine("        RETURN;");
        sb.AppendLine("    END\n");
    }

    sb.AppendLine("    BEGIN TRY");
    sb.AppendLine("        BEGIN TRANSACTION;\n");
    sb.Append("        INSERT INTO dbo." + tbl + "(");
    sb.Append(string.Join(",", nonPk.Select(x => "[" + x.FieldName +
"]")) + ",CreateBy");
    sb.AppendLine();
    sb.Append("        VALUES(");
    sb.Append(string.Join(",", nonPk.Select(x => "@" + x.FieldName)) +
",@UserId");
    sb.AppendLine();
    sb.AppendLine("        COMMIT TRANSACTION;\n        SET @" + pkName
+ " = SCOPE_IDENTITY();\n        SET @ResponseMSG = '' + input.TableName + "
Save Successfully';\n        SET @IsSuccess = 1;\n    END TRY\n");
    sb.AppendLine("    BEGIN CATCH\n        ROLLBACK TRANSACTION;
\n        IF ERROR_NUMBER() IN (2601,2627)\n            SET @ResponseMSG =
'Duplicate " + input.TableName + " Data';\n        ELSE\n
BEGIN\n            EXEC usp_ErrorLog @UserId;\n            SET
@ErrorNumber=(SELECT IDENT_CURRENT('tbl_SQLErrorLog'));\n            SET
@ResponseMSG = ERROR_MESSAGE();\n        END\n    END CATCH\nEND\nGO\n");

    // Update
    sb.AppendLine("CREATE PROCEDURE usp_Update" + input.TableName);
    sb.AppendLine("(");
    foreach (var c in nonPk)
        sb.AppendLine($"@{c.FieldName} {c.SqlType}{(c.IsNullable ? " =
NULL" : string.Empty)},");
    sb.AppendLine("@UserId INT ,");
    sb.AppendLine("@EntityId INT = NULL ,");
    sb.AppendLine($"@{pkName} INT ,");
    sb.AppendLine("@ResponseMSG NVARCHAR(254) = 'Invalid' OUTPUT ,");
    sb.AppendLine("@IsSuccess BIT = 0 OUTPUT, ");
    sb.AppendLine("@ErrorNumber INT = 0 OUTPUT");
    sb.AppendLine(")");
    sb.AppendLine("AS");
    sb.AppendLine("BEGIN");
    sb.AppendLine("    SET NOCOUNT ON;\n    EXEC sp_set_session_context
@key=N'UserId', @value=@UserId;\n");

    if (nonPk.Any(x => x.FieldName.Equals("Name",
StringComparison.OrdinalIgnoreCase)))
    {
        sb.AppendLine("        IF EXISTS(SELECT 1 FROM dbo." + tbl +

```

```

" (NOLOCK) T WHERE T.Name = @Name AND T." + pkName + " <> @" + pkName + ")");
        sb.AppendLine("        BEGIN");
        sb.AppendLine("        SET @IsSuccess = 0;");
        sb.AppendLine("        SET @ResponseMSG = '" + input.TableName
+ " already exists.'");
        sb.AppendLine("        RETURN;");
        sb.AppendLine("        END\n");
    }

    sb.AppendLine("    BEGIN TRY");
    sb.AppendLine("    BEGIN TRANSACTION;\n");
    sb.Append("    UPDATE TOP (1) dbo." + tbl + " SET ");
    sb.Append(string.Join(", ", nonPk.Select(x => "[" + x.FieldName + "]
= @" + x.FieldName)));
    sb.AppendLine(", ModifyBy = @UserId, UpdateLogDateTime = GETDATE()
WHERE " + pkName + " = @" + pkName + " ;\n");

    sb.AppendLine("    COMMIT TRANSACTION;\n        SET @ResponseMSG = '" +
input.TableName + " Update Successfully';\n        SET @IsSuccess = 1;\n    END
TRY\n");
    sb.AppendLine("    BEGIN CATCH\n        ROLLBACK TRANSACTION;
\n        IF ERROR_NUMBER() IN (2601,2627)\n            SET @ResponseMSG =
'Duplicate " + input.TableName + " Data';\n        ELSE\n
BEGIN\n            EXEC usp_ErrorLog @UserId;\n            SET
@ErrorNumber=(SELECT IDENT_CURRENT('tbl_SQLErrorLog'));\n            SET
@ResponseMSG = ERROR_MESSAGE();\n        END\n    END CATCH\nEND\nGO\n");

    // Delete
    sb.AppendLine("CREATE PROCEDURE usp_Del" + input.TableName +
"ById");
    sb.AppendLine("(");
    sb.AppendLine($"@{pkName} INT ,");
    sb.AppendLine("@UserId INT ,");
    sb.AppendLine("@EntityId INT = NULL ,");
    sb.AppendLine("@ResponseMSG NVARCHAR(254) = 'Invalid' OUTPUT ,");
    sb.AppendLine("@IsSuccess BIT = 0 OUTPUT, ");
    sb.AppendLine("@ErrorNumber INT = 0 OUTPUT");
    sb.AppendLine(")");
    sb.AppendLine("AS");
    sb.AppendLine("BEGIN");
    sb.AppendLine("    SET NOCOUNT ON;\n    EXEC sp_set_session_context
@key=N'UserId', @value=@UserId;\n");
    sb.AppendLine("    BEGIN TRY\n        BEGIN TRANSACTION;
\n        DELETE TR FROM dbo." + tbl + " (NOLOCK) TR WHERE TR." + pkName +
" = @" + pkName + " ;\n        COMMIT TRANSACTION;\n        SET @ResponseMSG =
'" + input.TableName + " Deleted Successfully';\n        SET @IsSuccess =
1;\n    END TRY\n");
    sb.AppendLine("    BEGIN CATCH\n        ROLLBACK TRANSACTION;

```

```

\n      IF ERROR_NUMBER() = 547\n          SET @ResponseMSG = 'Cannot
delete. ' + input.TableName + ' is in use.';\n      ELSE\nBEGIN\n      EXEC usp_ErrorLog @UserId;\n      SET
@ErrorNumber=(SELECT IDENT_CURRENT('tbl_SQLErrorLog'));\n      SET
@ResponseMSG = ERROR_MESSAGE();\n      END\n      END CATCH\nEND\nGO\n");

// GetAll
sb.AppendLine("CREATE PROCEDURE usp_GetAll" + input.TableName);
sb.AppendLine("(");
sb.AppendLine($"@{pkName} INT ,");
sb.AppendLine("@UserId INT ,");
sb.AppendLine("@EntityId INT = NULL");
sb.AppendLine(")");
sb.AppendLine("AS");
sb.AppendLine("BEGIN");
sb.AppendLine("    SET NOCOUNT ON;\n    EXEC sp_set_session_context
@key=N'UserId', @value=@UserId;\n");
sb.AppendLine("    SELECT TR." + pkName + ", " + string.Join(",",
nonPk.Select(x => "TR." + x.FieldName)) + " FROM dbo." + tbl + " (NOLOCK) TR;
\nEND\nGO\n");

// GetById
sb.AppendLine("CREATE PROCEDURE usp_Get" + input.TableName +
"ById");
sb.AppendLine("(");
sb.AppendLine($"@{pkName} INT ,");
sb.AppendLine("@UserId INT ,");
sb.AppendLine("@EntityId INT = NULL");
sb.AppendLine(")");
sb.AppendLine("AS");
sb.AppendLine("BEGIN");
sb.AppendLine("    SET NOCOUNT ON;\n    EXEC sp_set_session_context
@key=N'UserId', @value=@UserId;\n");
sb.AppendLine("    SELECT TOP (1) TR." + pkName + ", " +
string.Join(",", nonPk.Select(x => "TR." + x.FieldName)) + " FROM dbo." + tbl +
" (NOLOCK) TR WHERE TR." + pkName + " = @" + pkName + ";\nEND\nGO\n");

return sb.ToString();
}
#endregion

#region CSharp Builders
private string MapToCSharpType(string sql)
{
    var s = sql.ToLower();
    if (s.StartsWith("int")) return "int";
    if (s.StartsWith("nvarchar") || s.StartsWith("varchar") ||
s.StartsWith("char")) return "string";

```



```

        if (s.StartsWith("datetime")) return "DateTime?";
        if (s.StartsWith("bit")) return "bool?";
        if (s.StartsWith("decimal")) return "decimal?";
        return "string";
    }

    private string BuildModel(AutoCrudInput input)
    {
        var ns = "Dynamic.BE." + (string.IsNullOrEmpty(input.Module) ?
"Master" : input.Module);
        var sb = new StringBuilder();
        sb.AppendLine("using System;\nusing System.Collections.Generic;
\nusing Dynamic.DataAccess.Global;\n");
        sb.AppendLine("namespace " + ns);
        sb.AppendLine("{");
        sb.AppendLine("    public class " + input.TableName + " :
ResponseValues");
        sb.AppendLine("    {");

        var pk = input.Fields.FirstOrDefault(f => f.IsPrimaryKey);
        var pkName = pk != null ? pk.FieldName : input.TableName + "Id";
        sb.AppendLine("        public int? " + pkName + " { get; set; }");

        foreach (var f in input.Fields.Where(x => !x.IsPrimaryKey))
        {
            sb.AppendLine("        public " + MapToCSharpType(f.SqlType) +
" " + f.FieldName + " { get; set; }");
        }

        sb.AppendLine("        public int CUserId { get; set; }");
        sb.AppendLine("        public int EntityId { get; set; }");
        sb.AppendLine("    }\n");

        sb.AppendLine("    public class " + input.TableName +
"Collections : List<" + input.TableName + ">\n    {\n        public bool
IsSuccess { get; set; }\n        public string ResponseMSG { get; set; }\n    }
\n");

        sb.AppendLine("}");
        return sb.ToString();
    }

    private string BuildDA(AutoCrudInput input)
    {
        var ns = "Dynamic.DA." + (string.IsNullOrEmpty(input.Module) ?
"Master" : input.Module);
        var beNs = "Dynamic.BE." + (string.IsNullOrEmpty(input.Module) ?
"Master" : input.Module);
        var sb = new StringBuilder();

```

```

var pk = input.Fields.FirstOrDefault(f => f.IsPrimaryKey);
var pkName = pk != null ? pk.FieldName : input.TableName + "Id";
var nonPk = input.Fields.Where(x => !x.IsPrimaryKey).ToList();

sb.AppendLine("using System;\nusing System.Collections.Generic;
\nusing Dynamic.DataAccess.Global;\n");
sb.AppendLine("namespace " + ns);
sb.AppendLine("{");
sb.AppendLine("    internal class " + input.TableName + "DB");
sb.AppendLine("    {");
sb.AppendLine("        DataAccessLayer1 dal = null;\n");
sb.AppendLine("        public " + input.TableName + "DB(string
hostName, string dbName)\n        {\n            dal = new
DataAccessLayer1(hostName, dbName);\n        }\n");

// SaveUpdate
sb.AppendLine("        public ResponseValues SaveUpdate(" + beNs +
"." + input.TableName + " beData, bool isModify)\n        {\n
ResponseValues resVal = new ResponseValues();\n            dal.OpenConnection();
\n            System.Data.SqlClient.SqlCommand cmd = dal.GetCommand();
\n            cmd.CommandType = System.Data.CommandType.StoredProcedure;\n");

foreach (var c in nonPk)
{
    sb.AppendLine($"
cmd.Parameters.AddWithValue(\"@{c.FieldName}\", beData.{c.FieldName} ??
(object)DBNull.Value);");
}

sb.AppendLine("        cmd.Parameters.AddWithValue(\"@UserId\",
beData.CUserId);\n        cmd.Parameters.AddWithValue(\"@EntityId\",
beData.EntityId);\n        cmd.Parameters.AddWithValue(\"@" + pkName + "\",
beData." + pkName + ");\n");

sb.AppendLine("        if (isModify)\n
{\n            cmd.CommandText = \"usp_Update\" + input.TableName + \"\";
\n        }\n        else\n        {\n
cmd.Parameters[" + nonPk.Count + "].Direction =
System.Data.ParameterDirection.Output;\n            cmd.CommandText =
\"usp_Add\" + input.TableName + \"\";\n        }\n");

sb.AppendLine("        cmd.Parameters.Add(\"@ResponseMSG\",
System.Data.SqlDbType.NVarChar, 254).Direction =
System.Data.ParameterDirection.Output;\n
cmd.Parameters.Add(\"@IsSuccess\", System.Data.SqlDbType.Bit).Direction =
System.Data.ParameterDirection.Output;\n
cmd.Parameters.Add(\"@ErrorNumber\", System.Data.SqlDbType.Int).Direction =

```

```

System.Data.ParameterDirection.Output;\n");

        sb.AppendLine("                try\n                    {\n
cmd.ExecuteNonQuery();\n                    if (!isModify)\n
resVal.RId = Convert.ToInt32(cmd.Parameters["@" + pkName + "\"].Value);\n
\n                    resVal.ResponseMSG =
Convert.ToString(cmd.Parameters["@ResponseMSG"].Value);\n
resVal.IsSuccess = Convert.ToBoolean(cmd.Parameters["@IsSuccess"].Value);\n
\n                    }\n                    catch (Exception ex)\n
{\n                        resVal.IsSuccess = false;\n                            resVal.ResponseMSG
= ex.Message;\n                    }\n                    finally\n
{\n                        dal.CloseConnection();\n                            }\n\n                            return
resVal;\n                    }\n");

// Delete, GetAll, GetById methods (concise versions) - follow same
style

        sb.AppendLine("                public ResponseValues DeleteById(int UserId,
int EntityId, int " + pkName +
")\n                {\n                    ResponseValues resVal = new ResponseValues();
\n                    dal.OpenConnection();\n
System.Data.SqlClient.SqlCommand cmd = dal.GetCommand();\n
cmd.CommandType = System.Data.CommandType.StoredProcedure;\n
cmd.Parameters.AddWithValue("@UserId", UserId);\n
cmd.Parameters.AddWithValue("@EntityId", EntityId);\n
cmd.Parameters.AddWithValue("@" + pkName + "\", " + pkName + ");\n
cmd.CommandText = \"usp_Del\" + input.TableName + \"ById\";\n
cmd.Parameters.Add("@ResponseMSG", System.Data.SqlDbType.NVarChar,
254).Direction = System.Data.ParameterDirection.Output;\n
cmd.Parameters.Add("@IsSuccess", System.Data.SqlDbType.Bit).Direction =
System.Data.ParameterDirection.Output;\n
cmd.Parameters.Add("@ErrorNumber", System.Data.SqlDbType.Int).Direction =
System.Data.ParameterDirection.Output;\n
\n                    try\n
{\n                        cmd.ExecuteNonQuery();\n                            resVal.ResponseMSG =
Convert.ToString(cmd.Parameters["@ResponseMSG"].Value);\n
resVal.IsSuccess = Convert.ToBoolean(cmd.Parameters["@IsSuccess"].Value);\n
\n                    }\n                    catch (Exception ex)\n
{\n                        resVal.IsSuccess = false;\n                            resVal.ResponseMSG
= ex.Message;\n                    }\n                    finally\n
{\n                        dal.CloseConnection();\n                            }\n\n                            return
resVal;\n                    }\n");

// getAll

        sb.AppendLine("                public " + input.TableName + "Collections
getAll" + input.TableName + "(int UserId, int EntityId)\n                {\n
" + input.TableName + "Collections dataColl = new " + input.TableName +
"Collections();\n                    dal.OpenConnection();\n
System.Data.SqlClient.SqlCommand cmd = dal.GetCommand();\n
cmd.CommandType = System.Data.CommandType.StoredProcedure;\n

```

```

cmd.Parameters.AddWithValue(@"UserId", UserId);\n
cmd.Parameters.AddWithValue(@"EntityId", EntityId);\n
cmd.CommandText = @"usp_GetAll" + input.TableName + "";\n
try\n
    {\n
        System.Data.SqlClient.SqlDataReader reader =\n
cmd.ExecuteReader();\n
        while(reader.Read())\n
{\n
            var beData = new " + input.TableName + "();\n
            int ord = 0;\n
            if(!(reader[ord] is\n
DBNull)) beData." + pkName + " = reader.GetInt32(ord); ord++;}\n");

    // mapping for non-pk
    int ordIndex = 1;
    foreach (var c in nonPk)
    {
        var map = "if(!(reader[" + ordIndex + "] is DBNull)) beData." +\n
c.FieldName + " = ";
        if (c.SqlType.ToLower().StartsWith("int")) map +=\n
"reader.GetInt32(" + ordIndex + ");";
        else if (c.SqlType.ToLower().StartsWith("nvarchar") ||\n
c.SqlType.ToLower().StartsWith("varchar")) map += "reader.GetString(" +\n
ordIndex + ");";
        else map += "reader.GetValue(" + ordIndex + ").ToString()";
        sb.AppendLine("                " + map);\n
        ordIndex++;
    }

    sb.AppendLine("                dataColl.Add(beData);\n
\n                }\n
            reader.Close();\n
dataColl.IsSuccess = true;\n
            dataColl.ResponseMSG =\n
GLOBALMSG.SUCCESS;\n
            }\n
            catch(Exception ex)\n
{ dataColl.IsSuccess = false; dataColl.ResponseMSG = ex.Message; }\n
finally{ dal.CloseConnection(); }\n
            return dataColl;\n
        }\n");

    // getById
    sb.AppendLine("        public " + input.TableName + " get" +\n
input.TableName + "ById(int UserId, int EntityId, int " + pkName + ")\n
{\n
    var beData = new " + input.TableName + "();\n
dal.OpenConnection();\n
    System.Data.SqlClient.SqlCommand cmd =\n
dal.GetCommand();\n
    cmd.CommandType =\n
System.Data.CommandType.StoredProcedure;\n
cmd.Parameters.AddWithValue(@"" + pkName + "\", " + pkName + ");\n
cmd.Parameters.AddWithValue(@"UserId", UserId);\n
cmd.Parameters.AddWithValue(@"EntityId", EntityId);\n
cmd.CommandText = @"usp_Get" + input.TableName + "ById";\n
try\n
    {\n
        System.Data.SqlClient.SqlDataReader reader =\n
cmd.ExecuteReader();\n
        if(reader.Read())\n
{\n
            int ord = 0; if(!(reader[ord] is DBNull)) beData." +\n
pkName + " = reader.GetInt32(ord); ord++;}\n");

```

```

        ordIndex = 1;
        foreach (var c in nonPk)
        {
            var map = "if(!(reader[" + ordIndex + "] is DBNull)) beData." +
c.FieldName + " = ";
            if (c.SqlType.ToLower().StartsWith("int")) map +=
"reader.GetInt32(" + ordIndex + ");";
            else if (c.SqlType.ToLower().StartsWith("nvarchar") ||
c.SqlType.ToLower().StartsWith("varchar")) map += "reader.GetString(" +
ordIndex + ");";
            else map += "reader.GetValue(" + ordIndex + ").ToString();";
            sb.AppendLine("                " + map);
            ordIndex++;
        }

        sb.AppendLine("            }\n                reader.Close();\n\n
            beData.IsSuccess = true; beData.ResponseMSG =
GLOBALMSG.SUCCESS;\n                }\n                catch(Exception ex)
{ beData.IsSuccess = false; beData.ResponseMSG = ex.Message; }\n
finally{ dal.CloseConnection(); }\n                return beData;\n            }\n");

        sb.AppendLine("    }\n}");
        return sb.ToString();
    }

    private string BuildBL(AutoCrudInput input)
    {
        var sb = new StringBuilder();
        var ns = "Dynamic.BL." + (string.IsNullOrEmpty(input.Module) ?
"Master" : input.Module);
        var daNs = "Dynamic.DA." + (string.IsNullOrEmpty(input.Module) ?
"Master" : input.Module);
        var beNs = "Dynamic.BE." + (string.IsNullOrEmpty(input.Module) ?
"Master" : input.Module);
        var pk = input.Fields.FirstOrDefault(f => f.IsPrimaryKey);
        var pkName = pk != null ? pk.FieldName : input.TableName + "Id";

        sb.AppendLine("using Dynamic.DataAccess.Global;\n");
        sb.AppendLine("namespace " + ns);
        sb.AppendLine("{\n    public class " + input.TableName + "\n
{\n        " + daNs + "." + input.TableName + "DB db = null;\n        int
_UserId = 0;\n\n        public " + input.TableName + "(int UserId, string
hostName, string dbName)\n            {\n                this._UserId = UserId;
\n                db = new " + daNs + "." + input.TableName + "DB(hostName, dbName);
\n            }\n\n        public ResponseValues SaveFormData(" + beNs + "." +
input.TableName + " beData)\n            {\n                bool isModify = beData." +
pkName + " > 0;\n                ResponseValues isValid = IsValidData(ref beData,
isModify);\n                if(isValid.IsSuccess) return db.SaveUpdate(beData,

```

```

isModify));\n                else return isValid;\n                }\n\n                public " +
input.TableName + "Collections GetAll" + input.TableName + "(int EntityId)
\n                {\n                    return db.getAll" + input.TableName + "(_UserId,
EntityId);\n                }\n\n                public " + input.TableName + " Get" +
input.TableName + "ById(int EntityId, int Id)\n                {\n                    return
db.get" + input.TableName + "ById(_UserId, EntityId, Id);\n                }\n\n
public ResponseValues DeleteById(int EntityId, int Id)\n                {\n
return db.DeleteById(_UserId, EntityId, Id);\n                }\n\n                public
ResponseValues IsValidData(ref " + beNs + "." + input.TableName + " beData, bool
IsModify)\n                {\n                    ResponseValues resVal = new ResponseValues();
\n                    try\n                    {\n                        if(beData == null)
resVal.ResponseMSG = GLOBALMSG.NO_DATA_FOUND;\n                        else if(IsModify
&& beData." + pkName + " == 0) resVal.ResponseMSG = GLOBALMSG.INVALID_DATA + \"
For Modify\";\n                        else if(!IsModify && beData." + pkName + " != 0)
resVal.ResponseMSG = GLOBALMSG.INVALID_DATA + \" For Save\";\n
else if(beData.CUserId == 0) resVal.ResponseMSG = \"Invalid User for CRUD\";
\n                    );

                    if (input.Fields.Any(x => x.FieldName.Equals("Name",
StringComparison.OrdinalIgnoreCase)))
                    {
                        sb.AppendLine("                    else
if(string.IsNullOrEmpty(beData.Name)) resVal.ResponseMSG = \"Please! Enter Name
\";\n");
                    }

                    sb.AppendLine("                    else { resVal.IsSuccess = true;
resVal.ResponseMSG = \"Valid\"; }\n                    }\n                    catch(Exception
ex) { resVal.IsSuccess = false; resVal.ResponseMSG = ex.Message; }\n
return resVal;\n                }\n                }\n}");

                return sb.ToString();
            }

private string BuildController(AutoCrudInput input)
{
    var sb = new StringBuilder();
    var ns = "Dynamic.BL." + (string.IsNullOrEmpty(input.Module) ?
"Master" : input.Module);
    var beFull = "Dynamic.BE." + (string.IsNullOrEmpty(input.Module) ?
"Master" : input.Module) + "." + input.TableName;
    var pk = input.Fields.FirstOrDefault(f => f.IsPrimaryKey);
    var pkName = pk != null ? pk.FieldName : input.TableName + "Id";

    sb.AppendLine("using System;\nusing System.Web.Mvc;\nusing
Dynamic.DataAccess.Global;\n");
    sb.AppendLine("[Authorize(Roles=\"Admin\")]\npublic class " +
input.TableName + "GeneratorController : Controller\n{");

```

```

        // Generate action - accept JSON posted AutoCrudInput
        sb.AppendLine("    [HttpPost]\n    public ActionResult
Generate(AutoCrudGenerator.Models.AutoCrudInput input)\n    {\n        var res =
new ResponseValues();\n        try\n        {\n            var gen = new
AutoCrudGenerator();\n            var r = gen.GenerateAll(input);\n
\n\n            // write files\n            var outRel =
string.IsNullOrEmpty(input.OutputFolderRelative) ? "~/Generated/" :
input.OutputFolderRelative;\n            var physicalOut =
Server.MapPath(outRel);\n            if(!Directory.Exists(physicalOut))
Directory.CreateDirectory(physicalOut);\n\n            var entityDir =
Path.Combine(physicalOut, input.TableName);\n            if(!
Directory.Exists(entityDir)) Directory.CreateDirectory(entityDir);\n
\n\n            System.IO.File.WriteAllText(Path.Combine(entityDir,
input.TableName + ".BE.cs"), r.ModelCode);\n
System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName +
"DB.DA.cs"), r.DAcode);\n
System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName +
".BL.cs"), r.BLcode);\n
System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName +
"Controller.Generated.cs"), r.ControllerCode);\n
System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName +
".Index.cshtml"), r.ViewCode);\n
System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName +
"_Create.sql"), r.CreateTableSql);\n
System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName +
"_Procedures.sql"), r.ProceduresSql);\n\n            // Optionally execute SQL
if explicitly requested (disabled by default)
                if (input.ExecuteSql)
                {
                    // Execute r.CreateTableSql + r.ProceduresSql against DB - do
this only if you trust the source
                    // Example: use your connection string and run commands using
SqlConnection/SqlCommand
                }
\n            return Json(new { Data = r, IsSuccess = r.IsSuccess, Message =
r.Message });\n        }\n        catch(Exception ex)\n        {\n
return Json(new { Data = (object)null, IsSuccess = false, Message =
ex.Message });\n        }\n    }\n}");

        return sb.ToString();
    }

    private string BuildIndexView(AutoCrudInput input)
    {
        var sb = new StringBuilder();
        sb.AppendLine("@{ Layout = "~/Views/Shared/_Layout.cshtml"; }\n");
        sb.AppendLine("<div ng-controller=\"GeneratorCtrl\">\n

```

```

<h3>Generated CRUD: " + input.TableName + "</h3>\n    <p>Files written to /
Generated/" + input.TableName + "</p>\n</div>\n");
        return sb.ToString();
    }
    #endregion
}
}
}

```

### 3) Admin MVC Controller (invokes generator)

**File:** AutoCrudGenerator/Controllers/AutoCrudController.cs

The controller code above (generated by BuildController) is written into the generator project as a string. The project should also include a small admin UI or direct POST to Generate endpoint.

To keep the generator itself minimal, add this file that receives JSON and calls the generator (safe, admin-only):

```

using System;
using System.IO;
using System.Web.Mvc;
using AutoCrudGenerator.Models;

namespace AutoCrudGenerator.Controllers
{
    [Authorize(Roles = "Admin")]
    public class AutoCrudController : Controller
    {
        // POST: /AutoCrud/Generate
        [HttpPost]
        public ActionResult Generate(AutoCrudInput input)
        {
            var generator = new AutoCrudGenerator();
            var result = generator.GenerateAll(input);

            if (!result.IsSuccess)
                return Json(new { IsSuccess = false, Message =
result.Message });

            // Write files
            var outRel = string.IsNullOrEmpty(input.OutputFolderRelative) ? "~/
Generated/" : input.OutputFolderRelative;

```



```

        var physicalOut = Server.MapPath(outRel);
        if (!Directory.Exists(physicalOut))
Directory.CreateDirectory(physicalOut);

        var entityDir = Path.Combine(physicalOut, input.TableName);
        if (!Directory.Exists(entityDir))
Directory.CreateDirectory(entityDir);

        System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName
+ ".BE.cs"), result.ModelCode);
        System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName
+ "DB.DA.cs"), result.DAcode);
        System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName
+ ".BL.cs"), result.BLcode);
        System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName
+ "Controller.Generated.cs"), result.ControllerCode);
        System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName
+ ".Index.cshtml"), result.ViewCode);
        System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName
+ "_Create.sql"), result.CreateTableSql);
        System.IO.File.WriteAllText(Path.Combine(entityDir, input.TableName
+ "_Procedures.sql"), result.ProceduresSql);

        return Json(new { IsSuccess = true, Message = "Files generated.",
Path = "/Generated/" + input.TableName });
    }
}
}

```

## 4) Admin UI sample (AngularJS) — posts JSON to /AutoCrud/Generate

Place in Views/AutoCrud/Index.cshtml

```

<div ng-controller="GeneratorCtrl">
    <h3>Auto CRUD Generator (Admin)</h3>
    <div>
        Module: <input ng-model="meta.Module" />
        Table Name: <input ng-model="meta.TableName" />
        Use Prefix: <input type="checkbox" ng-model="meta.UseTablePrefix" />
    </div>
    <table>
        <thead><tr><th>Field</th><th>Type</th><th>Nullable</th><th>PK</th>
th><th>Unique</th><th></th></tr></thead>

```

```

<tbody>
  <tr ng-repeat="f in meta.Fields">
    <td><input ng-model="f.FieldName" /></td>
    <td><input ng-model="f.SqlType" /></td>
    <td><input type="checkbox" ng-model="f.IsNullable" /></td>
    <td><input type="checkbox" ng-model="f.IsPrimaryKey" /></td>
    <td><input type="checkbox" ng-model="f.IsUnique" /></td>
    <td><button ng-click="remove($index)">X</button></td>
  </tr>
</tbody>
</table>
<button ng-click="add()">Add Field</button>
<button ng-click="generate()">Generate</button>
<pre>{{resp | json}}</pre>
</div>

<script>
app.controller('GeneratorCtrl', function($scope,$http){
  $scope.meta = { Module:'Master', TableName:'Religion', UseTablePrefix:true,
Fields:
[{'FieldName':'Name',SqlType:'nvarchar(200)',IsNullable:false,IsPrimaryKey:false,IsUnique:true},
{'FieldName':'OrderNo',SqlType:'int',IsNullable:true,IsPrimaryKey:false}] };
  $scope.add = function(){
$scope.meta.Fields.push({'FieldName':'',SqlType:'nvarchar(100)',IsNullable:false,IsPrimaryKey:false});
  $scope.remove = function(i){ $scope.meta.Fields.splice(i,1); };
  $scope.generate = function(){ $http.post('/AutoCrud/Generate',
$scope.meta).then(function(r){ $scope.resp = r.data; alert(r.data.Message ||
'Done'); }, function(e){ $scope.resp = e.data; }); });
});
</script>

```

## 5) Security & Validation (must read)

- **Only admin users** should be able to access `/AutoCrud/Generate` — decorate controller with `[Authorize(Roles = "Admin")]`.
- **Server-side validation:** whitelist allowed SQL types (int, nvarchar(n), varchar(n), datetime, bit, decimal(18,2), float) and sanitize field names (letters, digits, underscore only). The generator uses a `SafeName` filter but you should add stricter checks.
- **Preview-first mode recommended:** Set `ExecuteSql = false` by default. Return SQL and files for admin review. Only execute SQL after explicit confirmation.
- **Backups:** run DB backup before applying generated DDL on production.

## 6) How to add to your solution

1. Create a new class library or add a new folder `AutoCrudGenerator` inside your web project.
  2. Add the model files ( `Models` folder) and `AutoCrudGenerator.cs`.
  3. Add `AutoCrudController` to `Controllers` folder.
  4. Add the Angular view to `Views/AutoCrud/Index.cshtml`.
  5. Build and run; go to the Admin UI and generate a sample `Religion`.
- 

## 7) Final notes

- I included thorough templates to match your existing `Religion` coding style and stored-procedure patterns (transactions, duplicate checks, `usp_ErrorLog` hooks). You can further customize templates in `AutoCrudGenerator`.
  - If you want, I can now paste the **complete literal files** for each code file (copy/paste-ready). Say **"Paste files"** and I'll dump every file here.
- 

*End of Auto-CRUD Generator document.*